

FinWallet Ana Spesifikasyonu

1. Ürün amacı

FinWallet; .NET 8 tabanlı, multi-currency bir digital-wallet side projectidir. Amaç gerçek finansal backend problemlerini göstermektir: transaction consistency, double-entry accounting, idempotency, concurrency, external integrations, fraud, authentication/session security, cutoff, campaigns, notifications, reconciliation, security ve observability.

2. Güncel kapsam

Customer/Auth

- TR/AZ country/phone policy ile registration.
- Redis tabanlı OTP verification.
- PBKDF2 password hashing.
- JWT access token + durable server session.
- Opaque refresh-token rotation ve reuse detection.

Wallet/Account

- TRY/USD/EUR wallet create/list.
- FakeBank üzerinden external BankAccount opening.
- Wallet ve BankAccount ayrı domain kavramlarıdır.
- v1'de FX conversion yoktur.

Financial Core

- Double-entry Ledger.
- Durable FinancialTransaction.
- MSSQL tabanlı durable idempotency.
- Atomik wallet-to-wallet transfer.
- Internal + external fraud decision.

Platform

- YARP Gateway tüm normal public/client ve FinWallet->provider HTTP trafiğinin giriş noktasıdır.
- Gateway JWT, internal-service authorization, rate limit, health checks ve load balancing uygular.
- Tüm Web API projelerinde Swagger vardır; production'da varsayılan kapalıdır.

3. Henüz tamamlanmamış kapsam

- Public BankDeposit ve BankWithdrawal.
- Merchant purchase / campaign accounting.
- Public Refund/Reversal flow.
- Durable FraudEvents/manual review.
- Transactional Outbox/Inbox.
- Transaction history/read model.
- ReconciliationRuns/ReconciliationIssues.

- Merkezi maskeli structured logging, OpenTelemetry ve alerting.
- Real MSSQL/Redis/YARP integration-concurrency test suite.
- Logout/session-revoke public endpoint.

4. Teknoloji kuralları

- .NET 8 / C# 12.
- ASP.NET Core controller-based Web API.
- MSSQL durable source of truth.
- Redis yalnız transient state.
- JWT auth; ASP.NET Core Identity kullanılmaz.
- YARP reverse proxy/gateway.
- Built-in DI.
- Paid/freemium NuGet yasaktır.
- Paket versiyonları Directory.Packages.props ile merkezi yönetilir.

5. Mimari

Ana uygulama modular monolith'tir:

```
FinWallet.Api
FinWallet.Application
FinWallet.Domain
FinWallet.Infrastructure
FinWallet.Shared.Contracts
FinWallet.Shared.Web
FinWallet.Gateway
```

External simulatorlar ayrı Web API prosesleridir:

```
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

6. Finansal doğruluk invariant'ları

- 1 Hiçbir para hareketi Ledger'ı atlayamaz.
- 2 Her posted journal için total Debit = total Credit olmalıdır.
- 3 MSSQL final financial consistency authority'dir.
- 4 Redis kaybı duplicate money, negative balance veya ledger corruption yaratmamalıdır.
- 5 External HTTP açık financial SQL transaction içinde çalıştırılmaz.
- 6 Duplicate command ve retry güvenli olmalıdır.
- 7 Same idempotency key + different payload conflict'tir.
- 8 Completed financial history mutate/delete edilmez; correction reversal/compensation ile yapılır.
- 9 Currency her Money değerinin parçasıdır ve commit öncesi doğrulanır.
- 10 Reconciliation mismatch'i sessizce balance update ederek düzeltmez.

7. Gateway güvenlik kuralı

```
Client -> Gateway -> FinWallet.Api
FinWallet.Api -> Gateway /providers/* -> Fake providers
```

- Protected public /api/* rotalarında Gateway JWT ister.
- FinWallet.Api JWT/ownership kontrolünü tekrar yapar.
- FinWallet.Api provider route'larına InternalServiceKey ile gider.
- Gateway destination request'e ayrı DownstreamServiceKey yazar.
- Backend/provider business endpointleri downstream key olmadan doğrudan çağrıyı reddeder.

8. Finansal işlem modeli

Wallet transfer sırası:

```
completed durable replay
-> durable session/risk signals
-> internal fraud
-> external fraud
-> final Allow
-> atomic MSSQL posting
```

Atomic posting aynı transaction'da idempotency + balances + FinancialTransaction + LedgerJournal + LedgerEntries commit eder.

9. Security kuralları

- Password/OTP/JWT/refresh token/service key/connection secret loglanmaz.
- SQL parameterized kullanılır.
- Auth/ownership client-supplied ID veya risk flag'e güvenmez.
- JWT signing algorithm code-level invariant'tır.
- PBKDF2 scheme değişikliği versioned migration gerektirir.
- Rate/body/header/connection/timeouts Gateway ve backend katmanlarında uygulanır.
- Volumetric DDoS için ayrıca ingress/WAF/cloud DDoS control gerekir.

10. Definition of Done

Bir feature ancak şu durumda tamamlanır:

- solution build başarılı;
- ilgili testler başarılı;
- financial/concurrency/idempotency invariant'ları korunmuş;
- external failure davranışı tanımlanmış;
- güvenlik/logging etkileri değerlendirilmiş;
- package inventory güncel;
- TR/EN XML documentation güncel;
- etkilenen tüm Markdown dokümanları TR+EN ve kodla tutarlı.

11. v1 kapsam dışı

- gerçek banka/fraud/SMS/email providerları;
- credit-card acquiring/payment gateway;
- full core banking;
- loans/credit scoring;
- stock/crypto trading;
- FX conversion;

- Event Sourcing;
- zorunlu Kafka/RabbitMQ;
- full microservice decomposition.

Teknik Analiz

1. Amaç

FinWallet'ın teknik hedefi; para hareketlerinin doğru, tekrar çalıştırılabilir, izlenebilir ve dış servis hatalarına dayanıklı olduğu bir finansal backend örneği oluşturmaktır. Sistem yalnız başarılı senaryoya değil; duplicate request, concurrency, provider timeout, fraud failure, session revoke ve veri tutarlılığı problemlerine göre tasarlanır.

2. Fonksiyonel kapsam

Güncel uygulanmış kapsam:

- customer registration + OTP verification;
- login, JWT access token, refresh-token rotation, server-side session;
- TRY/USD/EUR wallet create/list;
- external bank-account opening through FakeBank;
- internal/external fraud evaluation;
- durable idempotent wallet-to-wallet transfer;
- double-entry ledger;
- YARP Gateway üzerinden client ve provider trafiği;
- Swagger, rate limit ve ortak HTTP güvenlik kontrolleri.

Planlanan fakat henüz tamamlanmayan finansal kapsam:

- BankDeposit;
- BankWithdrawal;
- merchant purchase/campaign accounting;
- refund/reversal public flows;
- durable manual fraud review;
- outbox/inbox;
- reconciliation;
- transaction-history read model.

3. Kritik kalite gereksinimleri

****Financial correctness:**** Wallet balance, FinancialTransaction, IdempotencyRecord ve Ledger aynı finansal transaction sınırında tutarlı commit edilmelidir.

****Idempotency:**** Aynı para hareketi retry edildiğinde ikinci kez para hareketi oluşmamalıdır. Aynı key farklı payload ile kullanılırsa conflict oluşmalıdır.

****Concurrency:**** Aynı wallet'tan eş zamanlı harcamalar overspend yaratmamalıdır. MSSQL final authority'dir.

****Security:**** Public trafik Gateway JWT kontrolünden geçer; service-level JWT/ownership tekrar doğrulanır. Provider rotaları ayrı internal/downstream service credential kullanır.

****Availability:**** Provider timeout veya 5xx durumları finansal doğruluğu bozmamalıdır. Fraud gibi zorunlu kararlar fail-closed davranır.

****Auditability:**** Finansal hareketler ledger ve immutable business transaction kayıtlarıyla açıklanabilir olmalıdır.

4. Veri kaynakları

- ****MSSQL:**** müşteri, authentication/session, wallet, bank account, transaction, idempotency ve ledger için durable source of truth.
- ****Redis:**** OTP gibi transient state. Para doğruluğunun kaynağı değildir.
- ****Fake providers:**** Harici sistem davranışlarını simüle eder; FinWallet tablolarını doğrudan değiştiremez.

5. Runtime sınırları

Client -> YARP Gateway -> FinWallet.Api
FinWallet.Api -> YARP Gateway -> Fake providers

Gateway routing/security/traffic yönetir. Business authorization ve financial rules Gateway'e taşınmaz.

6. Performans yaklaşımı

- SqlConnection connection pooling kullanılır.
- Redis ConnectionMultiplexer singleton tutulur.
- HttpClientFactory + SocketsHttpHandler connection pooling kullanılır.
- YARP destination connection/timeouts ve load-balancing config'ten yönetilir.
- Finansal doğruluk için gerekli SQL isolation/locking yalnız benchmark uğruna zayıflatılmaz.

7. Test yaklaşımı

Moq yalnız Application orchestration sınırları için kullanılır. Gerçek MSSQL locking, Redis Lua ve YARP routing davranışları mock ile doğrulanmış sayılmaz; integration/concurrency testleri gerektirir.

8. Mevcut en önemli gap

Yeni wallet sıfır bakiye ile açılır ve public BankDeposit endpoint'i henüz yoktur. Bu nedenle public API kullanarak tamamen uçtan uca register -> fund -> transfer akışı henüz tamamlanamaz. Bu, projenin sıradaki en önemli fonksiyonel eksikliğidir.

Mimari

1. Mimari stil

Ana finansal uygulama ****modüler monolith**** olarak tutulur. Amaç Wallet, Transaction ve Ledger gibi aynı atomik para hareketine katılan bileşenleri erken microservice ayrıştırmasıyla distributed transaction problemine dönüştürmemektir.

Solution'ın ana projeleri:

```
src/
  FinWallet.Api
  FinWallet.Application
  FinWallet.Domain
  FinWallet.Infrastructure
  FinWallet.Shared.Contracts
  FinWallet.Shared.Web
  FinWallet.Gateway
simulators/
  FakeBank.Api
  FakeFraud.Api
  FakeCutoff.Api
  FakeCampaign.Api
  FakeCommunication.Api
tests/
  FinWallet.Application.Tests
```

2. Katman sorumlulukları

****Domain:**** Entity/value object/invariant/state transition ve finansal kurallar. HTTP, SQL veya provider DTO bilmez.

****Application:**** Use-case orchestration ve port/interface'ler. Domain'i kullanır; Infrastructure implementasyonlarına bağımlı değildir.

****Infrastructure:**** MSSQL, Redis, JWT token üretimi, provider HttpClient adapter'ları ve persistence implementasyonları.

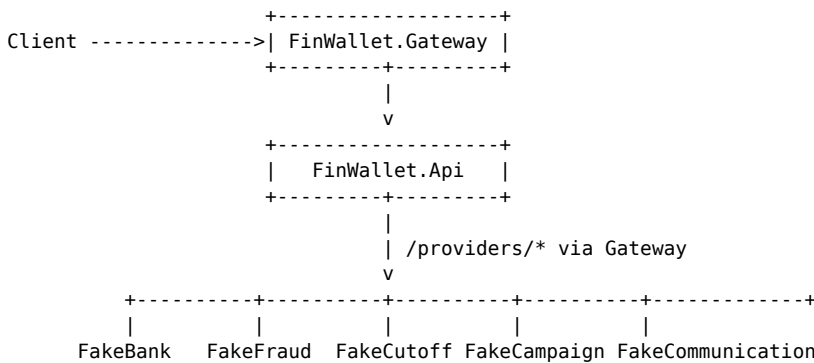
****Api:**** Controller, HTTP contract mapping, authentication/authorization composition ve exception mapping.

****Shared.Contracts:**** Ortak HTTP envelope ve simulatorlar arası paylaşılan stabil transport contractları.

****Shared.Web:**** Swagger, rate limit, Kestrel limits, CORS, security headers ve internal service-key kontrolü gibi ortak host davranışları.

****Gateway:**** YARP route/cluster, edge JWT, internal-service authorization, load balancing ve traffic controls.

3. Runtime topolojisi



Client normalde backend servislerini doğrudan çağırır. FinWallet.Api provider simulatorlarını da doğrudan çağırır; Gateway provider rotalarını kullanır.

4. Trust boundary'ler

- 1 ****Client -> Gateway:**** Public auth boundary. Protected /api/* rotalarında JWT gerekir.
- 2 ****Gateway -> FinWallet.Api:**** Gateway ayrı downstream service credential ekler. API ayrıca JWT ve ownership doğrular.
- 3 ****FinWallet.Api -> Gateway provider route:**** InternalServiceKey gerekir.
- 4 ****Gateway -> simulator:**** DownstreamServiceKey gerekir.

Bu yapı Gateway bypass edildiğinde business endpoint'in otomatik güvenilir hale gelmesini engeller.

5. Finansal transaction sınırı

Wallet-to-wallet transfer için aşağıdakiler tek MSSQL transaction içinde commit edilir:

- durable idempotency;
- source wallet debit;
- destination wallet credit;
- FinancialTransaction;
- LedgerJournal;
- LedgerEntries.

External HTTP bu SQL transaction açıkken çalıştırılmaz.

6. Bağımlılık yönü

Api -> Application -> Domain
Api -> Infrastructure -> Application/Domain
Gateway -> Shared.Web/Shared.Contracts
Simulators -> Shared.Web/Shared.Contracts

Domain'in Infrastructure/API'ye dependency alması yasaktır. Application provider-specific DTO veya SQL implementation bilmez.

7. Neden microservice değil?

Bugünkü sınırlar deployment bağımsızlığı için değil, doğruluk ve anlaşılabilirlik için seçilmiştir. Domain modülleri yeterince stabil olduğunda ve bağımsız scale/team ownership ihtiyacı olduğunda servis ayrıştırması değerlendirilebilir. Wallet/Ledger gibi aynı atomik commit'e ihtiyaç duyan bölümlerin ayrılması özellikle yüksek maliyetlidir.

Tasarım Desenleri

FinWallet tasarım desenlerini isim kullanmak için değil, belirli finansal problemlere çözüm oldukları yerde kullanır. Gereksiz abstraction veya interface katmanı eklenmemelidir.

DDD-lite

Entity, Value Object, Aggregate ve invariant kavramları finansal modelin sınırlarını açıklamak için kullanılır. Tam event-sourcing veya ağır DDD altyapısı yoktur.

Adapter + Anti-Corruption Layer

IBankProvider, IExternalFraudProvider ve ICommunicationGateway gibi Application portları provider-specific DTO/enumlari Domain'den ayırır. FakeBank/FakeFraud detayları Infrastructure içinde kalır.

Application Orchestrator

Registration, bank-account opening ve transfer gibi use-case'ler handler'lar tarafından sıralı şekilde koordine edilir. Controller business workflow taşımaz.

Chain of Responsibility / Rule Engine

Internal fraud değerlendirmesi bağımsız InternalFraudRule kurallarından oluşur. Yeni bir fraud kuralı mevcut handler'ı büyük ölçüde değiştirmeden eklenebilir.

Policy Pattern

Internal ve external fraud kararlarının birleşimi FraudDecisionPolicy gibi policy nesneleriyle açıkça tanımlanır.

Explicit Unit of Work / SQL Transaction Boundary

Wallet transfer için generic UnitOfWork abstraction yerine SqlWalletTransferPostingStore transaction sınırını açıkça sahiplenir. Locking ve commit sırası business correctness'in parçasıdır.

Idempotent Command

Para değiştiren request client tarafından Idempotency-Key taşır. MSSQL'deki durable record aynı key/same payload replay'ini güvenli yapar; same key/different payload conflict olur.

Optimistic / Compare-and-Set Concurrency

BankAccount gibi lifecycle state update'lerinde expected status + timestamp ile CAS kullanılır. Wallet transfer gibi yoğun para hareketinde gerekli yerde explicit locking/Serializable uygulanır.

Double-Entry Ledger

Her finansal etki Debit/Credit toplamı eşit bir journal ile temsil edilir. Correction geçmiş kaydı değiştirmek yerine reversal/compensation ile yapılır.

State Machine

Authentication/session, BankAccount ve FinancialTransaction lifecycle'ları yalnız izin verilen state geçişleri üzerinden ilerler.

Cache-Aside / TTL

Redis yalnız transient state için TTL tabanlı kullanılır. Financial source of truth değildir.

Timeout / Safe Retry

Provider çağrılarında timeout vardır. Retry yalnız operasyon idempotency açısından güvenliyse uygulanmalıdır; arbitrary financial POST otomatik retry edilmez.

Transactional Outbox / Inbox

Mimari olarak planlanmıştır fakat henüz production flow içinde uygulanmış değildir. Post-commit notification ve duplicate callback işleme için sıradaki desenlerden biridir.

Saga / Compensation

Uzun external-bank workflow'larında gerektiğinde kullanılacaktır. Tek DB içindeki wallet transfer için saga kullanılmaz; tek MSSQL transaction daha doğru ve daha basittir.

Reconciliation

Ledger/balance/provider statement uyumsuzlukları sessizce düzeltilmez; issue olarak tespit edilip izlenmelidir. Reconciliation altyapısı planlanmıştır.

CQRS-lite

Command/query kod organizasyonu ayrıştırılabilir; ayrı read database veya event-sourcing zorunlu değildir.

FinWallet API Rehberi

Public giriş noktası

Normal client yalnız Gateway'i çağırır:

`http://localhost:8080`

Ana business prefix: `/api/v1`.

Anonymous Gateway rotaları:

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/registration/verify`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/refresh`

Diğer `/api/*` rotaları Gateway'de JWT ister. FinWallet.Api `JWT/authorization/ownership` kontrolünü ayrıca tekrar yapar.

Ortak HTTP contract

Tüm success/failure body'leri `ServiceResult<T>` kullanır. Client logic sırası:

- 1 HTTP status;
- 2 stabil code.

Human-readable message business logic için parse edilmemelidir.

Correlation

Client X-Correlation-Id gönderebilir. Shared web platform yalnız bounded alphanumeric`/-/_` formatını kabul eder; geçersiz veya eksik değer yerine yeni ID üretir. Correlation ID transaction veya idempotency key değildir ve PII içermemelidir.

Authentication

Protected endpoint:

`Authorization: Bearer <access-token>`

JWT Gateway ve FinWallet.Api'de doğrulanır. Para değiştiren transfer akışı ayrıca JWT sid değerini durable CustomerSession ile kontrol eder.

Swagger

Development:

Gateway	<code>http://localhost:8080/swagger</code>
FinWallet.Api	<code>http://localhost:8081/swagger</code>
FakeBank.Api	<code>http://localhost:8082/swagger</code>
FakeFraud.Api	<code>http://localhost:8083/swagger</code>
FakeCutoff.Api	<code>http://localhost:8084/swagger</code>
FakeCampaign.Api	<code>http://localhost:8085/swagger</code>
FakeCommunication	<code>http://localhost:8086/swagger</code>

Production'da Swagger varsayılan kapalıdır.

Auth endpointleri

****Register**** — `POST /api/v1/auth/register` Pending customer oluşturur ve OTP'yi FakeCommunication'a gönderir. Başarı: HTTP 202.

****Verify**** — POST /api/v1/auth/registration/verify OTP verify+consume yapar ve customer'ı aktive eder.

****Login**** — POST /api/v1/auth/login Credential doğrular, durable session, access token ve refresh token üretir.

****Refresh**** — POST /api/v1/auth/refresh Opaque refresh token rotation yapar; token single-use'dur ve reuse detection session family revoke eder.

****Logout**** — henüz public endpoint olarak uygulanmadı.

Wallet endpointleri

****POST /api/v1/wallets**** — JWT gerekir.

```
{ "currency": "TRY" }
```

Desteklenen currency: TRY, USD, EUR. İlk create 201, aynı customer/currency tekrarında mevcut wallet 200 döner. Yeni wallet sıfır available/blocked balance ile başlar.

****GET /api/v1/wallets**** — JWT subject'e ait wallet'ları listeler.

Bank account

****POST /api/v1/bank-accounts**** — JWT gerekir.

```
{ "walletId": "<wallet-guid>" }
```

Akış:

```
Gateway JWT
-> API JWT + ownership
-> durable BankAccount(Opening)
-> SQL tamamlanır
-> Gateway /providers/bank/*
-> FakeBank
-> provider result validation
-> CAS state update
```

External HTTP açık SQL transaction içinde çalışmaz.

Wallet transfer

****POST /api/v1/transfers**** gerekir:

- JWT;
- active durable session;
- Idempotency-Key;
- owned/active source wallet;
- distinct active destination;
- aynı currency;
- positive/bounded amount;
- final fraud Allow;
- yeterli bakiye.

```
POST /api/v1/transfers
Authorization: Bearer <JWT>
Idempotency-Key: transfer-000001
Content-Type: application/json
```

```
{
  "sourceWalletId": "aaaaaaaa-1111-4111-8111-111111111111",
  "destinationWalletId": "bbbbbbbbb-2222-4222-8222-222222222222",
  "amount": 125.50
}
```

Sıra:

```
completed replay
-> session/risk
-> internal fraud
-> FakeFraud via Gateway
-> combined decision
-> atomic MSSQL posting
```

Completed replay aynı immutable transaction'ı döndürür; ikinci fraud evaluation veya ikinci money movement yapılmaz.

Funding durumu

Public BankDeposit endpoint'i henüz yoktur. Yeni wallet 0 bakiye ile açıldığı için register'dan sonra public endpointlerle kaynak wallet'ı fonlayıp transfer etmek mümkün değildir. Test fixture doğrudan balance UPDATE etmemeli; balanced FinancialTransaction + LedgerJournal/Entries oluşturmalıdır.

Internal/provider rotaları

```
/providers/bank/*
/providers/fraud/*
/providers/cutoff/*
/providers/campaign/*
/providers/communication/*
```

Bunlar public client API değildir. Gateway internal-service authorization ister; destination business endpoint ayrıca downstream service key doğrular.

Platform hata örnekleri

- JWT yok/geçersiz: 401 GATEWAY_UNAUTHORIZED.
- Direct backend without downstream key: 401 INTERNAL_SERVICE_UNAUTHORIZED.
- Rate limit: 429 RATE_LIMITED.
- Unsupported write content type: 415.
- TRACE/CONNECT: 405.
- Body/header limit aşımı: business processing başlamadan reddedilir.

Harici Entegrasyonlar

Entegrasyon ilkeleri

FinWallet simulator source code'ları aynı repository'de olsa bile her simulatorı external provider gibi ele alır.

Kurallar:

- provider servisleri controller-based ASP.NET Core API'dir;
- success/failure body'leri ServiceResult<T> kullanır;
- FinWallet provider storage'ını doğrudan okumaz/yazmaz;
- provider DTO/enumlari Infrastructure anti-corruption adapter'ları arkasında kalır;
- internal ve provider ID'leri ayrı tutulur;
- external HTTP financial SQL transaction açıkken çalışmaz;
- retry yalnız operation-level idempotency güvenliyse yapılır;
- normal provider çağrıları YARP Gateway üzerinden gider.

Gateway provider rotaları

```
/providers/bank/  
/providers/fraud/  
/providers/cutoff/  
/providers/campaign/  
/providers/communication/
```

Local default: http://localhost:8080/providers/....

FinWallet.Api Gateway'e X-Internal-Service-Key ile gider. Gateway bu credential'ı doğrular, destination request'te ayrı DownstreamServiceKey kullanır. Provider business endpoint'i downstream key yoksa doğrudan çağrışı reddeder.

HttpClient politikası

Provider adapterları HttpClientFactory + SocketsHttpHandler kullanır:

- provider-specific timeout;
- max connections per server;
- pooled connection lifetime/idle timeout;
- GZip/Deflate/Brotli decompression;
- cookies disabled;
- internal service header delegating handler.

Arbitrary financial POST'a otomatik retry eklenmez; timeout remote side-effect oluşmadığını kanıtlamaz.

FakeCommunication.Api

SMS/communication simulatorudur. Ana endpoint:

```
POST /api/v1/communication/sms
```

FinWallet route:

```
POST /providers/communication/api/v1/communication/sms
```

OTP body sensitive'dir ve production loglarına yazılmaz. Fake mode ile failure/delay/timeout senaryoları simüle edilebilir.

FakeBank.Api

Provider-side account/transaction state'ini sahiplenir; FinWallet Wallet/Ledger tablolarını deęiřtirmez.

Desteklenen provider kavramları:

- account opening/read/activation/pending;
- deposit/withdrawal-like transaction start/finalize/read;
- statement data for reconciliation;
- stable provider request key ile duplicate protection.

IBankProvider Application portudur; FakeBankProvider Infrastructure adapterıdır. Adapter provider envelope/enum/currency map eder, correlation taşıır ve unsafe automatic financial retry yapmaz.

FakeFraud.Api

Internal fraud kurallarından bağımsız external fraud provider simulatorıdır.

POST /providers/fraud/api/v1/fraud/evaluate

Request'e server-derived risk signals gider. Raw device ID yerine stable hashed reference kullanılır. Transfer için external fraud zorunludur; timeout/network/malformed response fail-closed davranır.

FakeCutoff.Api

Bank/currency/transaction-type için simulated business calendar/cutoff/settlement hesaplar. Özellikle BankDeposit ve BankWithdrawal akışları için planlanmıştır. Simulated holiday datası legal production source deęildir.

FakeCampaign.Api

Campaign eligibility/discount/sponsor calculation yapabilir. Muhasebe etkisini balanced ledger ile uygulama sorumluluęu FinWallet'a aittir.

Health ve load balancing

YARP clusterları configuration-driven destination ve PowerOfTwoChoices kullanır. Critical provider clusterlarında active health checks; ana FinWallet clusterında passive transport-failure health de vardır.

SSRF / unsafe upstream önleme

Client arbitrary provider URL gönderemez. Provider destination'ları server-owned YARP/config deęerleridir. Böylece normal request payload'ı uygulamayı user-controlled URL'e HTTP çağrısı yapan SSRF primitive'ine dönüřtüremez.

Production network kuralı

Application-level downstream key direct-bypass riskini azaltır; production'da backend/provider servisleri ayrıca network policy/ingress ile public routable olmamalıdır.

Teknolojiler ve NuGet Paketleri

Teknoloji baseline

Teknoloji	Versiyon/Politika	Amaç
.NET	8	Runtime ve application platform
ASP.NET Core	8	Controller-based Web API
C#	12	Uygulama dili
YARP	2.3.0	Reverse proxy, routing, health, load balancing
Swashbuckle	10.2.3	Swagger/OpenAPI üretimi ve UI
Microsoft.Data.SqlClient	7.0.2	MSSQL durable persistence
StackExchange.Redis	3.0.17	Transient Redis state
JwtBearer	8.0.29	Gateway/API JWT validation
xUnit v3	3.2.2	Unit test framework
Moq	4.20.72	Application boundary mockları

Paket politikası

Paid/freemium NuGet dependency kabul edilmez. Seçim sırası:

- 1 .NET/ASP.NET Core built-in capability;
- 2 uygun Microsoft-maintained package;
- 3 yalnız açık kaynak, ücretsiz ve lisansı uyumlu third-party package.

Tüm paket versiyonları Directory.Packages.props ile merkezi pinlenir.

JwtBearer

JWT validation Gateway ve FinWallet.Api'de kullanılır. Hand-written JWT parsing/signing güvenlik riski nedeniyle tercih edilmez. Signing algorithm code-level invariant olarak sabittir.

Microsoft.Data.SqlClient

Explicit async SQL, transaction, locking ve financial persistence için kullanılır. MSSQL durable financial authority'dir. Financial path'te SQL parameterized olmalıdır. Generic ORM/repository abstraction locking görünürlüğünü gizlememelidir.

StackExchange.Redis

OTP/transient distributed state için kullanılır. ConnectionMultiplexer singleton'dır. Redis Wallet/Ledger/idempotency financial source of truth olmaz.

Yarp.ReverseProxy

Gateway route matching, authorization policy, transform, cluster load balancing, health check ve transport tuning için kullanılır. Financial business rule içermez.

Swashbuckle.AspNetCore

Tüm Web API'lerde ortak Swagger generation/UI sağlar. FinWallet.Shared.Web üzerinden register edilir. Production'da varsayılan kapalıdır.

xUnit v3 + Moq

Application orchestration unit testleri için kullanılır. Moq, external provider/persistence boundary call'larını izole etmek için uygundur; gerçek MSSQL locking, Redis Lua veya YARP routing'i kanıtlamaz.

Framework-only kullanılan yetenekler

Ek paket olmadan:

- Kestrel limits;
- ASP.NET Core rate limiting;
- CORS;
- security headers middleware;
- HttpClientFactory/SocketsHttpHandler;
- PBKDF2/HMAC/SHA cryptography;
- CancellationToken/TimeProvider.

PBKDF2 notu

PBKDF2 V1 work factor loose appsettings tuning değildir. Mevcut persisted credential row'ları per-password iteration metadata saklamadığı için work factor değişimi versioned hash migration/rehash gerektirir.

Yeni paket onay kaydı

Her yeni package için şu bilgiler dokümanite edilmelidir: exact ID, version, license, owner project, purpose, neden built-in capability yeterli değil, alternatifler ve financial/security impact.

Veritabanı Tasarımı

Source of truth

MSSQL customer, authentication/session ve financial state için durable source of truth'tür. Redis MSSQL'nin alternatifi değildir ve para doğruluğunu tek başına garanti edemez.

Persistence explicit parameterized Microsoft.Data.SqlClient kullanır. Özellikle financial transaction/locking kodunda SQL transaction boundary görünür tutulur.

Authentication schema

database/001_authentication_schema.sql:

- Customers;
- CustomerCredentials;
- CustomerSessions;
- RefreshTokens.

Raw password ve raw refresh token saklanmaz.

Financial accounts schema

database/002_financial_accounts_schema.sql:

- Wallets;
- BankAccounts.

Wallet invariant'ları:

- (CustomerId, Currency) unique;
- non-negative available/blocked balances;
- customer ownership FK;
- durable lifecycle;
- DECIMAL(19,4) balance.

BankAccount invariant'ları:

- wallet başına tek BankAccount;
- internal ve provider AccountId ayrı;
- wallet/customer/currency composite relationship;
- provider account ID ve IBAN-like değer tutarlı;
- lifecycle update'leri expected status + UpdatedAt compare-and-set kullanır.

Ledger/transaction schema

database/003_ledger_transaction_schema.sql:

- LedgerAccounts;
- FinancialTransactions;
- IdempotencyRecords;
- LedgerJournals;
- LedgerEntries.

****FinancialTransaction**** business operation kaydıdır; Wallet row veya LedgerJournal ile aynı kavram değildir.

Stabil transaction type'ları:

- 1 WalletTransfer
- 2 BankDeposit
- 3 BankWithdrawal
- 4 Refund
- 5 Reversal

Lifecycle:

- 1 Processing
- 2 Completed
- 3 Failed
- 4 Reversed

Financial decimal kuralı

Financial amount DECIMAL(19,4) uyumlu olmalıdır:

- maksimum 4 decimal place;
- maksimum absolute amount 999999999999999.9999.

Application boundary DB overflow'a güvenmek yerine amount/balance kapasitesini SQL'e gitmeden önce doğrular.

Durable idempotency

Identity:

Scope + CustomerId + IdempotencyKey

Wallet-transfer scope: WALLET_TRANSFER.

Request fingerprint canonical source/destination/amount değerlerinden SHA-256 ile üretilir.

Davranış:

- same key + same payload -> wait/replay aynı transaction;
- same key + different payload -> conflict;
- final authority MSSQL;
- replay current wallet balance değil immutable transaction alanları döndürür.

Posting store Serializable isolation altında idempotency range'i UPDLOCK, HOLDLOCK ile korur.

Wallet transfer accounting

Wallet Liability ledger account code:

WALLET-LIABILITY:{walletId:N}

Muhasebe:

Debit	source wallet liability	amount
Credit	destination wallet liability	amount

Bu pure internal liability reclassification'dır.

Atomic posting — uygulanmış

SqlWalletTransferPostingStore aynı MSSQL transaction içinde:

```
BEGIN SERIALIZABLE
-> claim/replay idempotency
-> wallet row locks (deterministic GUID order)
-> validate ownership/status/currency/balance
-> calculate domain debit/credit
-> find/create ledger accounts
-> insert Processing FinancialTransaction
-> create/post LedgerJournal + entries
-> SQL aggregate Debit/Credit validation
-> update wallet balances
-> finalize transaction
-> finalize idempotency
-> COMMIT
```

Her exception commit öncesi bütün finansal state'i rollback eder.

Debit = Credit defense in depth

Domain LedgerJournal.Post(...) balance kontrolü yapar. Persistence ayrıca inserted entry toplamalarını SQL aggregate ile kontrol eder. En az iki entry, positive totals ve exact equality sağlanmadan commit edilmez.

Domain rehydration

Persisted entity state reflection ile private setter zorlanarak değil, kontrollü Restore factory'leriyle materialize edilir.

Redis state

Redis registration OTP gibi transient challenge state saklar. Wallet/Ledger/FinancialTransaction truth Redis'e taşınmaz.

Güncel durum

Public POST /api/v1/transfers endpoint'i ****uygulanmıştır**** ve yukarıdaki posting store'u Application fraud/session orchestration üzerinden kullanır. Önceki dokümandaki “public transfer endpoint yok” notu artık geçerli değildir.

Kalan database işleri

- BankDeposit/BankWithdrawal persistence workflow;
- FraudEvents/manual review;
- Outbox/Inbox;
- ReconciliationRuns/ReconciliationIssues;
- AuditEvents/operational logging store yaklaşımı;
- integration/concurrency test fixtures.

Finansal Akışlar

Genel kural

Hiçbir para hareketi Ledger'ı atlayamaz. Wallet current balance operasyonel state'tir; finansal hareketin açıklanabilir tarihi Ledger ve FinancialTransaction kayıtlarıdır.

1. Wallet-to-Wallet Transfer — uygulanmış

Endpoint: POST /api/v1/transfers.

Akış:

```
Gateway JWT
-> API JWT + durable session
-> completed idempotency replay
-> server-side risk signals
-> internal fraud
-> external FakeFraud via Gateway
-> final Allow
-> one MSSQL financial transaction
  -> idempotency
  -> source debit
  -> destination credit
  -> FinancialTransaction
  -> LedgerJournal + LedgerEntries
-> commit
```

Muhasebe:

```
Debit    Source Wallet Liability
Credit   Destination Wallet Liability
```

Aynı request/key replay edildiğinde para ikinci kez hareket etmez.

2. BankDeposit — planlanan sıradaki akış

Amaç external bank movement onaylandıktan sonra FinWallet wallet'ını fonlamaktır. Public endpoint henüz yoktur.

Önerilen lifecycle:

```
Created
-> Fraud/Cutoff if required
-> BankPending
-> BankConfirmed
-> LedgerPost
-> Completed
```

External bank confirmation olmadan wallet'a doğrudan bakiye eklenmemelidir. Deposit posting, provider-side asset/clearing hesabı ile customer wallet liability arasında balanced journal oluşturmalıdır.

3. BankWithdrawal — planlanan

Withdrawal uzun external workflow olduğu için available balance'ın önce block edilmesi gerekir.

Önerilen state:

```
Available -> Blocked -> external bank pending
              -> success: settle
              -> failure: release/compensate
```

SQL transaction external HTTP süresince açık tutulmaz.

4. Refund / Reversal — planlanan public flow

Refund business-level iadedir; reversal daha çok önceki finansal etkinin muhasebesel ters kayıdır. Orijinal transaction/journal silinmez veya mutata edilmez. Yeni FinancialTransaction + ters journal

oluşturulur.

5. Campaign / Merchant Purchase — planlanan

FakeCampaign eligibility/discount hesaplayabilir; fakat discount'ın ekonomik etkisinin customer/merchant/sponsor hesaplarına doğru yansımaları FinWallet ledger sorumluluğudur.

6. Fraud Review

Review kararı bugün para hareketi oluşturmaz. Durable FraudEvents/manual-review queue henüz uygulanmamıştır.

7. Cutoff

FakeCutoff bank/currency/transaction type için processing/settlement uygunluğunu hesaplar. Bu entegrasyon özellikle BankDeposit/BankWithdrawal için planlanmıştır.

8. Notification

Financial commit tamamlandıktan sonraki notification başarısızlığı para transaction'ını geri almamalıdır. Bu nedenle güvenilir notification için Outbox yaklaşımı planlanmıştır.

9. Reconciliation

Planlanan kontroller:

- wallet current state vs ledger-derived balance;
- internal bank-related transactions vs FakeBank statement;
- idempotency/transaction/ledger orphan kayıtları.

Mismatch sessiz UPDATE ile düzeltilmez; issue olarak kaydedilir.

10. Bugünkü demo sınırı

Yeni wallet sıfır bakiye ile açılır. Public BankDeposit olmadığı için sadece public endpointlerle yeni customer oluşturup kaynak wallet'ı fonlayarak transfer yapmak henüz mümkün değildir. Integration fixture balance + FinancialTransaction + balanced LedgerJournal/Entries'i atomik üretmelidir; sadece Wallets.AvailableBalance update etmek geçersizdir.

Güvenlik Tasarımı

Temel ilkeler

- Financial correctness, least privilege ve credential safety önceliklidir.
- Normal HTTP trafik YARP Gateway üzerinden geçer.
- Gateway auth, service/domain authorization'ın yerine geçmez.
- MSSQL durable authentication/financial source of truth'tür.
- Redis transient support state'tir.
- Production secret değerleri source control'da tutulmaz ve gerekli secret yoksa startup fail-fast olur.
- Password, OTP, JWT, refresh token, service key ve connection secret loglanmaz.
- Ownership/risk facts server-side türetilir.

Gateway trust modeli

****Client -> Gateway:**** Protected /api/* rotalarında JWT gerekir; register/verify/login/refresh anonymous'dur.

****Gateway -> FinWallet.Api:**** Gateway downstream service key taşır; API JWT ve ownership'i yeniden doğrular.

****FinWallet.Api -> Gateway /providers/*:**** InternalServiceKey gerekir.

****Gateway -> Fake provider:**** Ayı DownstreamServiceKey gerekir.

Health route'ları orchestrator/YARP health check için açıktır; Swagger exposure environment config ile yönetilir.

HTTP attack surface

FinWallet.Shared.Web:

- Kestrel server header kapalı;
- TRACE/CONNECT blocked;
- body-bearing POST/PUT/PATCH JSON zorunlu;
- body/header count/header size limits;
- request-header/keep-alive timeout;
- max concurrent connections;
- per-IP fixed-window rate limit;
- queue default 0;
- CORS allow-list;
- bounded correlation ID;
- no-store/no-cache;
- CSP, frame/MIME/referrer/permissions/cross-origin headers.

Bu kontroller L7 abuse/resource exhaustion azaltır; volumetric DDoS için cloud/edge/ingress/WAF gerekir.

Password

V1:

- PBKDF2-HMAC-SHA512;

- 220,000 iterations;
- 32-byte random salt;
- 64-byte hash;
- constant-time comparison;
- persisted hash version.

Iteration count loose runtime tuning değildir; future change versioned migration/rehash gerektirir.

JWT ve session

JWT fixed HMAC-SHA256 kullanır; minimal sub, sid, JTI, iat claim'leri taşır. Balance/IBAN/contact içermez. Issuer/audience/signing key config/secret store'dan gelir. Lifetime güvenli bir aralıkta config edilebilir. Signing algorithm config edilemez.

High-risk transfer sid değerini MSSQL CustomerSession ile doğrular; revoke edilmiş session kısa ömürlü JWT bitmeden de para hareketini durdurabilir.

Refresh token

Opaque random token client'a verilir; raw token DB'ye yazılmaz. Rotation compare-and-set ile single-use'dur. Consumed token reuse session/token family revoke eder.

OTP

Redis'te raw OTP yerine HMAC digest tutulur. Lua script atomic issue/attempt/consume davranışı sağlar. Redis unavailable ise verification fail-closed olur.

SQL injection

Financial/auth persistence parameterized SqlCommand kullanır. Request text dynamic SQL fragment olarak kullanılmaz.

BOLA / ownership

- customer identity JWT sub'dan gelir;
- owned resource query'leri customer boundary içerir;
- transfer source ownership fraud öncesi ve locked posting içinde tekrar doğrulanır;
- destination/currency/lifecycle commit öncesi revalidate edilir.

Fraud güvenliği

Client isNewDevice, known beneficiary, velocity veya 24h amount gibi trust flag gönderemez. Bu sinyaller durable server state'ten üretilir.

```
completed replay
-> server-side risk signals
-> internal fraud
-> external fraud
-> combined decision
-> posting only on Allow
```

External fraud unavailable/malformed ise fail-closed.

Idempotency / replay

Money-changing transfer Idempotency-Key ister. Durable identity Scope + CustomerId + Key; canonical request hash same-key/different-payload kullanımını conflict yapar.

Logging

Asla loglanmaz:

- password/hash/salt;
- OTP/digest/pepper;
- JWT/refresh token;
- Authorization header;
- internal/downstream keys;
- signing key;
- credential içeren SQL/Redis connection strings;
- unmasked phone/email/IBAN/account.

OWASP/API security özeti

- Broken Access Control/BOLA: gateway+service auth, owner-aware SQL.
- Authentication failures: JWT/session/refresh rotation/login lockout/OTP/rate limit.
- Injection: parameterized SQL.
- Security Misconfiguration: prod Swagger off, bounded HTTP config, no server header.
- Cryptographic failures: PBKDF2, HMAC, short JWT, secret management.
- Sensitive business flows: durable idempotency, fraud, ledger.
- Resource consumption: rate/body/header/connection/timeouts.
- SSRF/unsafe API consumption: provider URLs server-owned config'tir.
- Supply chain: central package versions, CI build/test; SBOM/vulnerability scan halen açık iştir.

Kalan security işleri

- Gateway bypass/rate limit integration tests;
- dependency vulnerability/SBOM scanning;
- centralized masked logging/SIEM/alerting;
- NetworkPolicy/ingress/TLS hardening;
- logout/session-revoke endpoint;
- durable FraudEvents/manual review;
- incident/reconciliation runbooks.

Test Stratejisi

Güncel durum

Gateway/platform hardening öncesinde solution'da test projesi ve Moq kullanımı yoktu. Artık tests/FinWallet.Application.Tests vardır ve:

- xUnit v3;
- Moq;
- .NET 8 / Microsoft Testing Platform

kullanır.

İlk strict-mock test OpenBankAccountHandler üzerinde IWalletStore, IBankAccountStore ve IBankProvider bağımlılıklarını mock eder; owned wallet bulunmazsa external bank provider'ın hiç çağrılmadığını doğrular.

Neyi mock etmeliyiz?

Application orchestration testlerinde davranışı testin konusu olmayan boundary'ler mock edilebilir:

- IBankProvider;
- IExternalFraudProvider;
- ICommunicationGateway;
- persistence interface'leri;
- zaman bağımlılığı için TimeProvider.

Financial orchestration'da strict mock tercih edilir; beklenmeyen provider/DB çağrısı önemli bug olabilir.

Mock ile kanıtlanamayacaklar

Aşağıdakiler gerçek infrastructure integration/concurrency test ister:

- MSSQL rollback;
- Serializable / UPDLOCK / HOLDLOCK;
- unique-key race;
- refresh-token compare-and-set;
- deterministic wallet lock ordering;
- idempotency range locking;
- persisted Debit=Credit aggregate check;
- connection pooling behavior;
- Redis Lua atomicity/reconnect/fail-closed;
- YARP route/auth/health/load balancing;
- Gateway direct-bypass rejection.

Gerekli integration suite

En az:

- 1 concurrent duplicate registration;
- 2 concurrent refresh rotation;

- 3 aynı Idempotency-Key ile 10 transfer;
- 4 same key + different payload;
- 5 tek source wallet'tan 100 concurrent debit;
- 6 A->B ve B->A opposite-direction deadlock testi;
- 7 OTP verify sırasında Redis unavailable;
- 8 transfer öncesi FakeFraud timeout;
- 9 Gateway JWT yok;
- 10 backend downstream key yok;
- 11 rate-limit rejection;
- 12 ledger imbalance injection -> rollback.

Finansal invariant testi

Load test yalnız HTTP success count'a güvenmez:

```
Initial balances
+ durable credits
- durable debits
= final balances
```

```
SUM(Ledger Debit) = SUM(Ledger Credit)
```

Ayrıca successful FinancialTransactions, IdempotencyRecords ve LedgerJournals reconcile etmelidir.

CI policy

CI:

```
restore
-> Release build --warnaserror
-> dotnet test --no-build
```

çalıştırır. Yeni project FinWallet.sln içine eklenmeden feature tamamlanmış sayılmaz.

Mevcut coverage yorumu

Unit-test altyapısı artık vardır ancak coverage halen başlangıç seviyesindedir. “Moq eklendi” gerçek MSSQL/Redis/YARP doğruluğu kanıtlandı anlamına gelmez.

Kod Dokümantasyon Standardı

Amaç

FinWallet codebase'inde business intent, financial invariant ve teknik davranışın hem Türkçe hem İngilizce anlaşılabilmesi için C# XML documentation çift dilli tutulur.

Zorunlu kapsam

Uygun olan her manually-written declaration dokümente edilir:

- class/record/struct/enum/interface;
- constructor;
- method;
- property;
- generic type parameter;
- method parameter;
- return value;
- anlamlı thrown exception.

Generated code hariçtir.

Format

Her XML summary'de Türkçe önce, İngilizce sonra:

```
/// <summary>
/// TR: Cüzdanın kullanılabilir bakiyesini temsil eder.
/// EN: Represents the wallet's available balance.
/// </summary>
```

Method/param/returns açıklamaları isim tekrarı yapmak yerine intent ve constraint anlatmalıdır.

Financial code beklentisi

İlgili yerde dokümantasyon şu invariant'ları açıklamalıdır:

- currency consistency;
- Debit = Credit;
- append-only/reversal davranışı;
- idempotency;
- concurrency/locking assumptions;
- allowed state transitions;
- source-of-truth ownership;
- external HTTP / transaction boundary.

Compiler enforcement

Directory.Build.props XML documentation üretimini açar ve CS1591 warning'ini error yapar. Bu externally visible member'lar için compiler-level enforcement sağlar. Review standardı internal/private production member'larda da gerekli açıklamayı ister.

Markdown standardı

docs/**/*.*.md ve root README.md Türkçe + İngilizce olmalıdır. Tercih edilen yapı:

```
## Türkçe  
...  
---
```

Agent ve Codex Geliştirme Akışı

Amaç

Bu belge, FinWallet geliştirmesinde Codex veya başka coding agent'ların nasıl scope alacağı, branch açacağı, doğrulama yapacağı, PR oluşturacağı ve review edileceğini tanımlar. Repository dosyaları, AGENTS.md, dokümanlar, ADR'ler ve GitHub issue/PR'ları source of truth'tür; chat geçmişi repository dokümantasyonunun yerine geçmez.

Çalışma modeli

Her agent:

- 1 bounded bir task/issue alır;
- 2 repository kurallarını okur;
- 3 current main'den agent/<task> branch'i açar;
- 4 gerekli kod/test/doc değişikliğini yapar;
- 5 Release build/test ile doğrular;
- 6 review edilebilir PR açar.

Önerilen roller:

- Solution Architect;
- Security/Auth;
- Financial Domain;
- Persistence/Concurrency;
- Integration;
- QA/Chaos;
- Code Review;
- Documentation.

Source-of-truth sırası

Çakışma olduğunda:

- 1 current task acceptance criteria;
- 2 AGENTS.md;
- 3 master specification;
- 4 accepted ADR;
- 5 architecture/security/database/API docs;
- 6 mevcut implementation convention;
- 7 agent varsayımı.

Agent üst seviye kuralı sessizce override edemez.

Branch/commit

Branch: agent/<bounded-task-name>.

Commit küçük ve cohesive olmalıdır. implement everything, fix stuff gibi belirsiz commitlerden kaçınılır. Financial behavior, package change, refactor ve docs mümkünse ayrı anlaşılır commitlerdir.

Coding agent sorumlulukları

- financial invariant'ları koru;
- class/interface/method/property için gerekli TR/EN XML docs'u yaz;
- paid/freemium package ekleme;
- yeni package inventory'yi güncelle;
- changed behavior için test ekle/güncelle;
- API/architecture/database/security docs'u güncelle;
- external HTTP'yi financial SQL transaction içine alma;
- idempotency/concurrency garantisini bozma;
- Redis'i financial source of truth yapma;
- tüm Markdown dokümanlarında TR+EN senkronunu koru.

Review agent kontrolü

- architecture dependency violations;
- ledger/financial correctness;
- double-spend/lost update;
- idempotency gaps;
- auth/OTP/token weakness;
- sensitive logging;
- unsafe retry/timeout;
- missing cancellation;
- over-engineering;
- undocumented package;
- missing TR/EN code/doc documentation.

QA/Chaos senaryoları

- simultaneous overspend;
- repeated idempotency key;
- same key + altered payload;
- Redis outage;
- provider timeout/500/slow;
- fraud unavailable;
- repeated refund/reversal;
- ledger mismatch;
- refresh token replay;
- OTP brute force;
- direct Gateway bypass;
- rate-limit/resource-exhaustion.

PR akışı

- 1 Branch'te bounded iş tamamlanır.
- 2 Diff self-review edilir.
- 3 Docs/test güncellenir.
- 4 dotnet restore, Release dotnet build --warnaserror, dotnet test çalıştırılır.
- 5 Draft PR açılır.
- 6 Review bulguları küçük commitlerle kapatılır.
- 7 CI green olur.
- 8 Financial/security sensitive PR final review alır.
- 9 Acceptance criteria karşılanınca merge edilir.

Parallel agent kuralı

Domain boundary, DB/ledger rule, API contract, auth ve error/idempotency convention stabil olmadan aynı feature alanında paralel agent'lar kendi sözleşmelerini invent etmemelidir. File conflict düşük ve contract stabil ise paralel çalışma uygundur.

DoD

Agent-generated code ancak acceptance criteria, build, relevant test, financial/security invariant, TR/EN XML docs, package inventory ve affected TR/EN Markdown docs tamamlandığında done sayılır.

Proje Yönetimi ve Teslimat Yol Haritası

Amaç

GitHub issue'ları authoritative work item'dır. Uygulama agent/* branch'lerinde geliştirilir ve PR ile main'e alınır.

Board modeli

Önerilen kolonlar:

- 1 Backlog
- 2 Ready
- 3 In Progress
- 4 In Review
- 5 Blocked
- 6 Done

Project board yoksa eşdeğer status:* label kullanılabilir.

Milestone'lar

Milestone	Kapsam	Güncel durum
M0 Architecture Baseline	scope, architecture, security, docs	Büyük ölçüde tamamlandı
M1 Foundation	solution/domain foundations	Tamamlandı
M2 Identity & Registration	OTP, password, JWT, refresh/session	Ana akış tamamlandı; logout açık
M3 Persistence & Concurrency	MSSQL, Redis, idempotency	Transfer için ana temel tamamlandı; outbox/inbox açık
M4 Financial Core	ledger, transaction, reversal model	Ledger + transfer tamamlandı; public reversal/refund açık
M5 External Providers	FakeBank/Fraud/Cutoff/Campaign/Communication	Simulator baseline tamamlandı
M6 Financial Flows	deposit, withdrawal, transfer, purchase, refund	Wallet transfer tamamlandı; diğerleri açık
M7 Reconciliation & Hardening	reconciliation, observability, chaos/integration tests	Açık
M8 Gateway/Platform	YARP, Swagger, rate-limit, security baseline	Tamamlandı

Güncel tamamlanan ana çalışmalar

- registration/auth/session baseline;
- Wallet/BankAccount persistence;
- Fake provider APIs;
- double-entry ledger schema/domain;
- durable idempotent wallet transfer;
- fraud orchestration;
- YARP Gateway + downstream trust boundary;
- Swagger tüm Web API'lerde;

- appsettings-based platform/performance tuning;
- xUnit v3 + Moq başlangıç test projesi;
- Release build + test CI;
- çift dilli dokümantasyon standardı.

Açık öncelikler

- 1 BankDeposit.
- 2 BankWithdrawal + cutoff.
- 3 Real MSSQL/Redis/YARP integration/concurrency tests.
- 4 Outbox/Inbox + reliable notification.
- 5 Durable FraudEvents/manual review.
- 6 Transaction history/read model.
- 7 Refund/Reversal public flows.
- 8 Reconciliation.
- 9 Centralized masked logging/OpenTelemetry/alerting.
- 10 Production deployment hardening.

Roller

- Solution Architect: boundaries/ADR/dependency direction.
- Security/Auth: auth/session/OTP/token controls.
- Financial Domain: wallet/ledger/transaction/accounting.
- Persistence/Concurrency: SQL/Redis/idempotency/locking.
- Integration: provider adapters/resilience/contracts.
- QA/Chaos: concurrency/duplication/failure scenarios.
- Code Review: architecture/security/financial correctness.
- Documentation: TR+EN docs ve code-doc sync.

PR workflow

- 1 Issue/acceptance criteria belirlenir.
- 2 Current main'den bounded branch açılır.
- 3 Kod + test + docs aynı feature scope içinde güncellenir.
- 4 Release build/test yapılır.
- 5 Draft PR açılır.
- 6 Review/CI bulguları kapatılır.
- 7 DoD sağlanınca merge edilir.

DoD

Code, test, TR/EN XML docs, affected TR/EN Markdown docs, security impact, failure behavior, concurrency/idempotency ve package inventory tamamlanmadan issue Done olmaz.

Wallet Transfer Akışı

Endpoint

POST /api/v1/transfers

Gereksinimler:

- valid JWT;
- JWT sid ile eşleşen active durable session;
- Idempotency-Key;
- JWT sub customer'a ait source wallet;
- distinct destination wallet;
- positive DECIMAL(19,4) uyumlu amount;
- source/destination currency eşitliği;
- final fraud Allow;
- yeterli source balance.

```
{
  "sourceWalletId": "11111111-1111-1111-1111-111111111111",
  "destinationWalletId": "22222222-2222-2222-2222-222222222222",
  "amount": 125.50
}
```

Currency client'tan trusted alınmaz; wallet state'ten türetilir.

Idempotency sırası

Completed replay fraud'dan ****önce**** kontrol edilir:

```
request
-> completed replay lookup
  -> same key + same immutable payload: original result
  -> same key + different payload: conflict
-> durable session/risk
-> internal fraud
-> external fraud
-> atomic posting
```

Bu precheck semantic/performance guard'dır. Final correctness yine atomic posting store'un Serializable idempotency locking'ine aittir.

Session validation

Valid JWT tek başına para hareketi için yeterli değildir. sid:

- aynı customer'a ait olmalı;
- revoked olmamalı;
- expired olmamalı;
- Active customer session olmalıdır.

Server-derived fraud signals

Client risk flag göndermez. SqlWalletTransferRiskSignalStore server state'ten:

- customer country;
- wallet currency;

- device reference/history;
- new-device;
- 5-minute transfer velocity;
- 24-hour same-currency amount;
- known-beneficiary

üretir.

Raw DeviceId provider'a gönderilmez; stable SHA-256 reference kullanılır.

Fraud flow

Internal Deny external Allow ile override edilemez. Internal Allow/Review sonrası FakeFraud gerekir. External timeout/network/malformed response fail-closed'dur.

Internal + External -> FraudDecisionPolicy -> Allow / Review / Deny

- Allow -> posting;
- Review -> no money movement;
- Deny -> no money movement;
- provider unavailable -> no money movement.

Durable manual-review queue henüz yoktur.

Atomic posting

Final Allow sonrası tek MSSQL transaction:

- IdempotencyRecord;
- source debit;
- destination credit;
- FinancialTransaction;
- Wallet Liability LedgerAccounts;
- balanced LedgerJournal + LedgerEntries;
- persisted SQL Debit/Credit equality.

External HTTP bu transaction içinde çalışmaz.

Double-entry

Debit	source wallet liability	amount
Credit	destination wallet liability	amount

Domain ve SQL seviyesinde journal balance kontrol edilir.

Replay response

Replay yalnız immutable alanları döndürür:

- FinancialTransaction ID;
- source/destination wallet IDs;
- amount/currency;
- original completion time;
- replay flag.

Current wallet balance replay body'ye konmaz; daha sonra deęişmiş olabilir.

Gateway, Swagger ve Platform Güvenliği

Topoloji

```
Client
-> FinWallet.Gateway :8080
  -> FinWallet.Api :8081
    -> Gateway /providers/bank/* -> FakeBank.Api :8082
    -> Gateway /providers/fraud/* -> FakeFraud.Api :8083
    -> Gateway /providers/cutoff/* -> FakeCutoff.Api :8084
    -> Gateway /providers/campaign/* -> FakeCampaign.Api :8085
    -> Gateway /providers/communication/* -> FakeCommunication.Api :8086
```

FinWallet provider simulatorlarını doğrudan çağırmaz; provider base URL'leri Gateway'e döner.

Customer authentication boundary

Gateway'de anonymous route'lar yalnız register/verify/login/refresh'tir. Diğer /api/* route'ları GatewayAuthenticated policy ile JWT ister. FinWallet.Api JWT ve business ownership kontrolünü tekrar yapar.

Internal provider boundary

FinWallet.Api /providers/* için X-Internal-Service-Key gönderir. Gateway InternalService policy ile doğrular ve proxied request'e ayrı DownstreamServiceKey yazar. Destination business endpoint downstream key olmadan 401 döner.

Bu iki ayrı trust boundary sağlar:

- 1 FinWallet -> Gateway;
- 2 Gateway -> destination.

YARP route/load balancing

Cluster config'i appsettings'tedir. PowerOfTwoChoices, destination address, health interval/timeout, passive reactivation, max connections ve activity timeout source code değiştirmeden override edilebilir. Development tek destination içerebilir; production replica ekleyebilir.

Rate limit / resource exhaustion

Repository seviyesinde:

- Gateway per-IP fixed-window rate limit;
- backend second-layer rate limit;
- queue default 0;
- request body/header count/header bytes limit;
- request-header/keep-alive timeout;
- max concurrent connections;
- provider-specific YARP body limits;
- provider HttpClient timeout/connection limits.

Bu L7 overload/abuse korumasıdır. Volumetric DDoS için cloud DDoS protection, L4/L7 LB, WAF/CDN/bot protection ve network policy gerekir.

Swagger

Swagger şu projelerde ortak FinWallet.Shared.Web ile vardır:

- Gateway;
- FinWallet.Api;
- FakeBank;
- FakeFraud;
- FakeCutoff;
- FakeCampaign;
- FakeCommunication.

Development'da açık, production'da varsayılan kapalıdır. Swagger görünürlüğü endpoint authorization'ı bypass etmez.

Shared HTTP security baseline

- Kestrel Server header off;
- TRACE/CONNECT blocked;
- JSON write content enforcement;
- correlation ID validation/regeneration;
- Cache-Control: no-store;
- X-Content-Type-Options: nosniff;
- X-Frame-Options: DENY;
- restrictive Referrer/Permissions/Cross-Origin policies;
- CSP;
- CORS allow-list;
- rate limiting;
- request/header/body/connection bounds.

OWASP/API mapping

****Broken Access Control/BOLA:**** Gateway+API JWT, owner-aware SQL, server-derived sub/sid, internal provider credentials.

****Security Misconfiguration:**** prod Swagger off, bounded HTTP config, no server header, separate env config.

****Supply Chain:**** central NuGet versions, explicit packages, CI restore/build/test. SBOM/vulnerability scan açık iştir.

****Cryptography:**** PBKDF2, salts, HMAC OTP, short JWT, secret-store, no credential logging.

****Injection:**** parameterized SQL, typed DTO, user-controlled SQL fragment yok.

****Sensitive Business Flows:**** double-entry, durable idempotency, server-side fraud, fail-closed provider behavior.

****Authentication Failures:**** JWT+session+refresh rotation+lockout+OTP+rate limit.

****Integrity Failures:**** DB constraints, append-only/reversal model, atomic posting.

****Resource Consumption:**** layered rate/body/header/connection/time limits.

****SSRF/Unsafe API:**** provider URLs server-owned config; external result provider adapter/validation arkasındadır.

Configuration modeli

Standard .NET precedence:

- 1 appsettings.json;
- 2 appsettings.{Environment}.json;
- 3 environment variables / secret injection;
- 4 ilerde eklenen explicit providerlar.

Network/pool/timeout/rate gibi operasyonel değerler config'tir. Crypto algorithm, double-entry invariant ve persistence semantics runtime switch değildir.

İlk Çalıştırma Happy Path: Kayıttan Wallet Transfer'a

Bu doküman projeyi ilk kez gören biri içindir. Normal client çağrılarının tamamı YARP Gateway'e gider.

Base URL

```
{{gateway}} = http://localhost:8080
```

FinWallet.Api :8081 normal client flow için doğrudan çağrılmamalıdır.

Örnek değişkenler:

```
{{tokenA}} Customer A JWT  
{{tokenB}} Customer B JWT  
{{walletA}} Customer A TRY wallet ID  
{{walletB}} Customer B TRY wallet ID
```

1. Customer A register

```
POST {{gateway}}/api/v1/auth/register  
Content-Type: application/json  
X-Correlation-Id: demo-register-a
```

```
{  
  "countryCode": "TR",  
  "phoneNumber": "+905321111111",  
  "email": "customer.a@example.test",  
  "password": "Example-Password-A-123!"  
}
```

Beklenen: HTTP 202 ve registration pending response. customerId saklanır.

****OTP notu:**** FinWallet registration response içinde OTP döndürmez. FakeCommunication raw OTP'yi simulated SMS body olarak alır. Local test/debug instrumentation ile okunmalıdır; public API'ye OTP leak endpoint eklenmemelidir.

2. Customer A OTP verify

```
POST {{gateway}}/api/v1/auth/registration/verify  
Content-Type: application/json
```

```
{  
  "customerId": "<customer-a-id>",  
  "code": "123456"  
}
```

Beklenen: HTTP 200 ve registration verified.

3. Customer A login

```
POST {{gateway}}/api/v1/auth/login  
Content-Type: application/json
```

```
{  
  "phoneNumber": "+905321111111",  
  "password": "Example-Password-A-123!",  
  "deviceId": "demo-device-a"  
}
```

Response içindeki accessToken -> tokenA olarak saklanır. Token loglanmaz.

4. Customer A TRY wallet create

```
POST {{gateway}}/api/v1/wallets  
Authorization: Bearer {{tokenA}}  
Content-Type: application/json
```

```
{ "currency": "TRY" }
```

İlk çağrı HTTP 201 WALLET_CREATED; aynı customer/currency tekrarıda HTTP 200 WALLET_EXISTS.
walletId -> walletA. Başlangıç available/blocked balance = 0.

5. Customer A external bank account

Wallet-to-wallet transfer için zorunlu değildir ama intended onboarding story'nin parçasıdır.

```
POST {{gateway}}/api/v1/bank-accounts
Authorization: Bearer {{tokenA}}
Content-Type: application/json

{ "walletId": "{{walletA}}" }
```

Provider pending ise HTTP 202 dönebilir. Retry aynı durable BankAccount ID'den deterministic provider request key ürettiği için duplicate account açmamalıdır.

6. Customer B register + verify + login

Aynı 1-3 adımlarını farklı phone/email/device ile tekrarla. Örnek phone: +905322222222. Access token -> tokenB.

7. Customer B TRY wallet

```
POST {{gateway}}/api/v1/wallets
Authorization: Bearer {{tokenB}}
Content-Type: application/json

{ "currency": "TRY" }
```

Wallet ID -> walletB.

8. Wallet list kontrolü

```
GET {{gateway}}/api/v1/wallets
Authorization: Bearer {{tokenA}}
```

JWT yok/geçersiz ise Gateway request'i backend'e göndermeden reddeder.

9. Funding prerequisite — mevcut gap

Yeni wallet 0 bakiye ile açılır. Public BankDeposit/funding endpoint'i ****henüz yoktur****. Dolayısıyla yalnız public API ile register -> fund -> successful transfer tamamlanamaz.

Şunu yapma:

```
UPDATE Wallets SET AvailableBalance = ...
```

Bu Ledger'ı atlayarak para yaratır ve reconciliation'ı bozar.

Transfer demo/test için controlled integration fixture aynı atomik transaction içinde balance + FinancialTransaction + balanced LedgerJournal/Entries oluşturmalıdır. Sıradaki gerçek feature BankDeposit olmalıdır.

10. Wallet transfer

walletA geçerli şekilde fonlandıktan sonra:

```
POST {{gateway}}/api/v1/transfers
Authorization: Bearer {{tokenA}}
Idempotency-Key: demo-transfer-a-to-b-0001
Content-Type: application/json
X-Correlation-Id: demo-transfer-a-to-b

{
  "sourceWalletId": "{{walletA}}",
  "destinationWalletId": "{{walletB}}",
  "amount": 125.50
}
```

Beklenen success: HTTP 200 WALLET_TRANSFER_COMPLETED, immutable transactionId, amount/currency/completedAt ve wasReplay=false.

11. Safe replay

Aynı request + aynı Idempotency-Key tekrar gönderilir. Beklenen:

- ikinci money movement yok;
- ikinci ledger posting yok;
- completed replay için ikinci fraud evaluation yok;
- same transaction ID;
- WALLET_TRANSFER_REPLAYED;
- wasReplay=true.

Aynı key farklı amount/destination ile gönderilirse conflict olur.

Gateway beklentileri

Request	Beklenen
Register/verify/login/refresh JWT olmadan	Gateway geçirir
Wallet/bank-account/transfer JWT olmadan	Gateway reddeder
/providers/* internal key olmadan	Gateway reddeder
Backend business endpoint downstream key olmadan	Destination reddeder
Rate/body/header limit aşımı	Business processing öncesi reddedilir

Swagger

http://localhost:8080/swagger Gateway
http://localhost:8081/swagger FinWallet.Api
http://localhost:8082/swagger FakeBank
http://localhost:8083/swagger FakeFraud

Normal business çağrısı yine Gateway :8080 üzerinden yapılır.

AI Destekli Mimari Karar Anlatımı

1. Başlangıç problemi

Side project generic CRUD yerine şu birleşimi seçti:

Digital Wallet + Double-entry Ledger + Fraud Detection + External Banking Integration

Amaç concurrency, idempotency, provider failure, accounting, session security, auditability ve reconciliation gibi gerçek financial-backend problemlerini göstermekti.

2. Neden modular monolith?

Wallet/Ledger/Transaction aynı atomik para hareketine katıldığı için başlangıçta microservice ayrıştırması distributed consistency maliyetini gereksiz erken getirdi. Domain modülleri logical olarak ayrıdır; deployment sınırı gerektiğinde sonra değerlendirilebilir.

3. Neden MSSQL financial source of truth?

ACID transaction, constraint, FK, unique key, explicit locking ve deterministic reconciliation query ihtiyacı vardır. Redis hızlı transient state için uygundur ama para var/yok kararının authority'si değildir.

4. Neden auth önce?

Wallet, BankAccount ve Transfer ownership kontrolü stabil customer/session identity ister. Bu yüzden registration -> OTP -> password -> login -> JWT -> refresh/session sırası money endpointlerinden önce kuruldu.

5. Neden JWT + server-side session?

JWT request authentication sağlar fakat tek başına immediate revoke sağlamaz. Minimal sid claim ve durable CustomerSession high-risk flow'da revoke/device/session kontrolü sağlar.

6. Neden Wallet ve BankAccount ayrı?

Wallet internal customer liability/balance state'tir. BankAccount external provider relationship'idir. Lifecycle ve source of truth farklı olduğu için aynı aggregate yapılmadı.

7. Neden FakeBank ayrı API?

External integration problemlerini gerçekten simüle etmek için HTTP boundary zorunlu tutuldu: timeout, 5xx, pending, provider-generated ID, idempotency ve polling. FinWallet provider DB'sine erişmez.

8. Neden durable state bank HTTP'den önce?

Bank-account opening önce internal BankAccount(Opening) kaydeder, SQL'i kapatır, sonra provider çağırır. Durable internal ID deterministic provider request key üretir; lost response duplicate external account'a dönüşmez.

9. Neden ledger transfer endpointinden önce?

Balance yalnız "şu an ne kadar var?" sorusunu cevaplar. Ledger "bu balance neden var?" sorusunun geçmişini sağlar. Bu nedenle money endpoint ledger invariant oluşmadan açılmaz.

10. FinancialTransaction neden LedgerJournal'dan ayrı?

FinancialTransaction business operation/lifecycle'dir; LedgerJournal accounting effect'tir. Failure/reversal/idempotency/resource identity ve reconciliation için ayrı kavramlar daha temizdir.

11. Neden explicit transfer store?

SqlWalletTransferPostingStore idempotency, iki wallet update'i, FinancialTransaction ve Ledger'ı tek MSSQL transaction'da sahiplenir. Bu bölgede locking/commit sırası correctness'in parçası olduğu için generic repository abstraction yerine explicit SQL tercih edildi.

12. Neden durable idempotency MSSQL'de?

Mobile timeout, gateway timeout, duplicate click veya retry aynı financial request'i tekrar gönderebilir. Final duplicate guarantee process cache/Redis'e bırakılmaz. Scope + CustomerId + Key + canonical payload hash kullanılır.

13. Neden fraud SQL money transaction'dan önce?

External provider HTTP uzun ve belirsizdir. Fraud önce değerlendirilir; yalnız final Allow sonrası kısa atomic financial SQL transaction açılır. Required fraud unavailable ise fail-closed.

14. Neden completed replay fraud'dan önce?

Dün completed olmuş aynı request bugün değişen fraud rule'ları yüzünden retry'da Deny olmamalıdır. Completed immutable result önce replay edilir.

15. Neden controller-based API?

Enterprise Web API convention, attributes, Swagger ve tutarlı action contractları için tüm HTTP servisleri controller standardında tutuldu; Minimal API ile iki convention karıştırılmadı.

16. ServiceResult neden yalnız HTTP contract?

ServiceResult<T> client response standardıdır. Domain/Application'ın HTTP envelope'a dependency alması business model ile transport katmanını gereksiz bağlardı.

17. YARP neden financial core'dan sonra?

Routing infrastructure yanlış para modelini düzeltmez. Auth, persistence, ledger ve transfer correctness kurulduktan sonra Gateway edge/platform boundary olarak eklendi.

18. Neden service-to-service de Gateway?

FinWallet provider adapterları /providers/* kullanır. İki credential vardır:

```
FinWallet -> Gateway    InternalServiceKey
Gateway -> Backend      DownstreamServiceKey
```

Bu Gateway'i yalnız client convention değil enforceable trust boundary yapar.

19. Neden Shared.Web?

Yedi Web API'ye ayrı ayrı Swagger/rate/CORS/header kodu kopyalamak configuration drift yarattı. FinWallet.Shared.Web yalnız host cross-cutting concern'leri merkezileştirir; business framework olmaz.

20. Neden bazı değerler config, bazıları değil?

Adres, timeout, pool, rate limit, body/header/connection limit, CORS, Swagger ve YARP destination operasyonel olduğu için config'tir. HS256 algorithm, PBKDF2 V1 migration semantics, Debit=Credit, financial decimal ve durable idempotency semantics correctness/security invariant olduğu için arbitrary runtime toggle değildir.

21. Neden Redis general cache olmadı?

Redis current use-case'te OTP/transient state için yeterlidir. Wallet balances veya final idempotency truth Redis'e taşınmadı; gereksiz cache invalidation ve correctness riski yaratılmadı.

22. Neden mock var ama yeterli sayılmıyor?

xUnit + Moq Application orchestration call davranışını test eder. SQL Serializable/range lock veya YARP route correctness gerçek infrastructure test ister.

23. Uygulama sırası

- scope/invariants
 - > architecture boundaries
 - > registration/auth/session
 - > communication/fraud providers
 - > wallet/ledger domain
 - > FakeBank + BankAccount
 - > persistence
 - > wallet APIs
 - > financial transaction/idempotency schema
 - > atomic transfer store
 - > fraud/session transfer orchestration
 - > YARP Gateway
 - > Shared.Web + Swagger/security
 - > config/performance tuning
 - > xUnit/Moq + CI tests
 - > bilingual/current documentation

24. Sıradaki doğal sıra

- 1 BankDeposit;
- 2 BankWithdrawal + cutoff;
- 3 integration/concurrency tests;
- 4 Outbox/Inbox + notification;
- 5 FraudEvents/manual review;
- 6 transaction history;
- 7 refund/reversal;
- 8 reconciliation;
- 9 OpenTelemetry/masked logging/operations.

25. Temel kural

Kolay endpoint'i, altındaki zor invariant hazır olmadan açma.

Transfer ledger/idempotency olmadan; bank flow durable state olmadan; provider retry idempotency olmadan expose edilmez.

MSSQL, Redis, HTTP ve Gateway Performans İncelemesi

Özet

Performans değişiklikleri financial correctness'i zayıflatmadan uygulanmalıdır. Operasyonel tuning değerleri mümkün olduğunda configuration-driven'dir; accounting/locking invariant'ları benchmark uğruna gevşetilmez.

MSSQL connection pooling

SqlConnection pooling connection string ile açıktır. Development baseline:

- Pooling=True;
- Min Pool Size=5;
- Max Pool Size=100;
- bounded connect timeout;
- Load Balance Timeout;
- application name.

SqlConnectionFactory short-lived logical connection üretir; dispose fiziksel connection'ı pool'a döndürür. Production pool size ölçülmüş DB kapasitesine göre ayarlanmalıdır.

Financial transaction performansı

Wallet transfer aynı transaction içinde source/destination balances, idempotency, FinancialTransaction ve Ledger'ı commit eder. Lock duration önemlidir ama correctness'ten daha önemli değildir. Wallet row'ları deterministic GUID order ile lock edilir; opposite-direction deadlock riskini azaltır.

Isolation level yalnız throughput artırmak için zayıflatılmaz.

Index yaklaşımı

Mevcut unique/index yapıları hot path'lerin önemli bölümünü destekler. Financial write-heavy tablolara speculative index eklemek insert/update/log maliyetini artırabilir. Yeni index Query Store, execution plan, logical read ve measured p95/p99 kanıtına göre eklenmelidir.

İzlenecek DB metrikleri:

- pool exhaustion;
- login/transfer DB p95/p99;
- deadlock;
- lock wait;
- top logical reads;
- Query Store regression;
- transaction-log growth;
- index usage/fragmentation.

Redis

Tek ConnectionMultiplexer singleton tutulur. Per-request multiplexer socket/performance regression olur.

Configurable değerler:

- AbortOnConnectFail;
- connect retry;
- connect timeout;
- sync timeout;
- keep-alive;
- client name.

OTP security fail-closed davranışı performans için gevşetilmez. Redis financial authority değildir. İzlenecekler: reconnect, latency, timeout, CPU, memory, eviction, network, Lua latency.

HttpClient

Provider client'ları HttpClientFactory + SocketsHttpHandler kullanır:

- pooled connection lifetime;
- pooled idle timeout;
- max connections/server;
- GZip/Deflate/Brotli;
- provider-specific timeout;
- cookies disabled.

Provider çağrıları YARP Gateway'e gider. Financial POST'lara endpoint-specific idempotency kanıtı olmadan otomatik retry eklenmez.

YARP

Clusterlar PowerOfTwoChoices kullanır. Production destination replica sayısı config ile artırılabilir. Performance controls:

- load balancing;
- active health;
- main cluster passive health;
- max destination connections;
- activity timeout;
- HTTP version policy;
- response buffering policy;
- pre-proxy request-size limit.

Rate limiting

Gateway public limit backend'den daha sıkıdır. Backend ikinci katman bypass/misrouting/internal runaway korumasıdır. Queue default 0; overload sırasında request biriktirmek yerine 429 ile hızlı reddetmek tail latency/memory baskısını azaltır.

Bilinçli yapılmayan optimizasyonlar

- Wallet balance'ı Redis source of truth yapmak yok.
- Financial SQL isolation'ı körlemesine düşürmek yok.
- Arbitrary financial POST retry yok.
- Authenticated financial response caching yok.

- Unbounded DB/HTTP connection pool yok.
- Fraud check'i latency için bypass etmek yok.
- Reconciliation'ı bozacak ledger denormalization yok.

Benchmark planı

Ölç:

- 1 registration RPS;
- 2 login p50/p95/p99;
- 3 wallet-list p50/p95/p99;
- 4 transfer p50/p95/p99;
- 5 same-wallet concurrent transfer throughput;
- 6 Gateway overhead vs controlled direct internal baseline;
- 7 Redis OTP latency;
- 8 SQL locks/deadlocks;
- 9 CPU/memory/GC/socket;
- 10 load sonrası ledger/balance/idempotency reconciliation.

Final balances/ledger reconcile etmeyen benchmark sonucu geçersizdir.

Son Teknik İnceleme

İnceleme özeti

Bu belge, Gateway/security/platform merge'lerinden sonraki mevcut main mimarisini değerlendirir. Son tam solution CI doğrulamasında restore, Release build (--warnaserror) ve dotnet test başarılı olmuştur. Bu, derleme ve mevcut unit-test kapsamının yeşil olduğunu gösterir; gerçek altyapı concurrency testlerinin tamamlandığı anlamına gelmez.

Güçlü taraflar

- Finansal source of truth MSSQL'dir; Redis para doğruluğunun kaynağı değildir.
- Wallet transfer balance/idempotency/transaction/ledger state'ini tek SQL transaction içinde commit eder.
- Double-entry Debit = Credit hem Domain hem persisted SQL seviyesinde kontrol edilir.
- Durable idempotency duplicate money movement'i engeller.
- External HTTP finansal SQL transaction dışında çalışır.
- Fraud risk sinyalleri client'tan değil server state'ten türetilir.
- JWT + durable session yaklaşımı high-risk işlemlerde revoke kontrolü sağlar.
- YARP Gateway public auth, internal routing, rate limit, health ve load balancing sınırı oluşturur.
- Backend/provider business endpointleri downstream service credential ile direct-bypass'a karşı korunur.
- Swagger tüm Web API'lerde standarttır ve production'da varsayılan kapalıdır.
- Merkezi package management ve warnings-as-errors kullanılır.
- xUnit v3 + Moq altyapısı vardır ve CI test çalıştırır.

Önceki isteğin karşılanma durumu

- YARP Gateway: ****tamamlandı****.
- Servis trafiğinin Gateway üzerinden geçirilmesi: ****uygulandı****; FinWallet provider adapterları /providers/* kullanır.
- Gateway JWT enforcement: ****tamamlandı****.
- Load balancing / health / rate limit / request limits: ****tamamlandı****.
- Tüm API'lere Swagger: ****tamamlandı****.
- OWASP/API abuse hardening: ****uygulama seviyesinde tamamlandı****; volumetric DDoS için edge/ingress/WAF koruması deployment sorumluluğudur.
- MSSQL/Redis/HttpClient/YARP performance review: ****tamamlandı****.
- Parametrik operasyon/tuning değerlerinin appsettings'e taşınması: ****tamamlandı****.
- Unit test mock kontrolü: önce eksikti; ****xUnit + Moq eklendi****.
- Register'dan transfera happy-path dokümanı: ****tamamlandı****, fakat public funding eksikliği açıkça belirtilmiştir.
- AI mimari karar anlatımı: ****tamamlandı****.
- Tüm maintained proje dokümanlarının TR+EN olması: ****tamamlandı****.

Bilinçli olarak appsettings'e taşınmayan değerler

Kullanıcının “parametrik tüm elementler” talebi operasyonel/tuning değerleri için uygulanmıştır. Aşağıdaki değerler bilinçli olarak runtime switch değildir:

- JWT imza algoritması;
- PBKDF2 V1 scheme/work factor migration semantics;
- double-entry eşitliği;
- financial decimal invariants;
- durable idempotency semantics;
- transaction/locking correctness rules.

Bu değerlerin config ile serbestçe değiştirilmesi güvenlik veya mevcut verinin doğrulanabilirliğini bozabilir.

Açık teknik borçlar

- 1 Public BankDeposit ve BankWithdrawal.
- 2 Durable FraudEvents/manual review.
- 3 Outbox/Inbox ve güvenilir post-commit notification.
- 4 ReconciliationRuns/ReconciliationIssues.
- 5 Transaction-history/read model.
- 6 Centralized masked structured logging + OpenTelemetry/alerting.
- 7 Gerçek MSSQL/Redis/YARP integration ve concurrency test suite.
- 8 Logout/session-revoke public endpoint.
- 9 Dependency vulnerability/SBOM pipeline.
- 10 Production ingress/WAF/network-policy/TLS deployment hardening.

Öncelik önerisi

En mantıklı sıra:

- 1 BankDeposit;
- 2 BankWithdrawal + cutoff;
- 3 integration/concurrency tests;
- 4 Outbox/Inbox + notification;
- 5 FraudEvents/manual review;
- 6 transaction history;
- 7 reconciliation;
- 8 telemetry/operations hardening.

Son karar

Mevcut sistem side-project amacı için sağlam bir financial-core/gateway baseline'idir. “Production-ready banking platform” olarak değerlendirilmemelidir; funding/withdrawal, reconciliation, observability ve gerçek altyapı concurrency testleri tamamlanmadan bu iddia doğru olmaz.

Docker Çalıştırma Rehberi

Amaç

Bu belge FinWallet çözümündeki tüm servisleri Docker Compose ile ilk kez ayağa kaldırmak, durumlarını kontrol etmek, logları incelemek, veritabanı/Redis volume'larını yönetmek, tek servis rebuild etmek ve güvenli şekilde durdurmak için operasyon rehberidir.

Normal Docker topolojisi:

```
Host
|
| http://localhost:8080
v
FinWallet.Gateway
|
+--> FinWallet.Api
|
+--> FakeBank.Api
+--> FakeFraud.Api
+--> FakeCutoff.Api
+--> FakeCampaign.Api
+--> FakeCommunication.Api

FinWallet.Api
|
+--> MSSQL
+--> Redis
+--> Gateway /providers/*
```

Normal kullanımda host'a yalnız Gateway portu publish edilir. MSSQL, Redis, FinWallet.Api ve fake provider portları Docker network içinde kalır. compose.debug.yml yalnız local debugging amacıyla bu servisleri 127.0.0.1 üzerinden host'a açar.

Docker dosyaları

Dosya	Amaç
compose.yml	Tüm uygulama ve infrastructure servislerinin ana Compose tanımı.
compose.debug.yml	MSSQL, Redis, FinWallet.Api ve fake provider portlarını yalnız localhost'a açan debug overlay.
compose.production.yml	Production-like environment için Swagger kapatma ve restart policy gibi ek hardening override'ları.
docker/Dockerfile.webapi	Gateway, FinWallet.Api ve tüm fake Web API projeleri için ortak multi-stage .NET 8 image build dosyası.
docker/mssql/init-db.sh	MSSQL ayağa kalktıktan sonra 001, 002, 003 schema scriptlerini sıralı ve version kontrollü çalıştırır.
.env.example	Local Compose environment değişkenleri için şablon. Gerçek secret içermez.
.dockerignore	Docker build context'inden bin/obj/log/test/docs/local-secret gibi gereksiz dosyaları çıkarır.
.gitignore	bin/obj/Debug/Release/log/secret/local Docker state gibi Git'e girmemesi gereken dosyaları engeller.

Docker servisleri

Service	Görev	Normal host portu
gateway	YARP edge gateway, JWT/routing/rate limit/load balancing	8080
finwallet-api	Ana FinWallet Web API	publish edilmez

Service	Görev	Normal host portu
fake-bank	Bank simulator	publish edilmez
fake-fraud	Fraud simulator	publish edilmez
fake-cutoff	Cutoff/business-calendar simulator	publish edilmez
fake-campaign	Campaign simulator	publish edilmez
fake-communication	SMS/Email simulator	publish edilmez
mssql	Financial source of truth	publish edilmez
mssql-init	One-shot schema initialization container	port yok
redis	OTP/transient support state	publish edilmez

Persistent volume'lar

Volume	İçerik	docker compose down sonrası kalır mı?
finwallet_mssql_data	SQL Server data/log dosyaları	Evet
finwallet_mssql_backup	SQL backup alanı	Evet
finwallet_redis_data	Redis AOF/RDB persistence	Evet

Uygulama container'ları stateless'tir. Uygulama logları named volume'a yazılmaz; container stdout/stderr üzerinden Docker log driver'a gider ve log rotation uygulanır.

1. Ön koşulları kontrol et

```
docker --version
docker compose version
```

Ne yapar:

- Docker Engine/Desktop CLI'nin kullanılabilir olduğunu doğrular.
- Compose v2 plugin'in kurulu olduğunu doğrular.
- Komut hata veriyorsa projeyi çalıştırmadan önce Docker Desktop/Engine problemi çözülmelidir.

Windows Docker Desktop kullanıyorsan Linux containers modunda çalıştır.

2. Repository root dizinine geç

```
cd finwallet
```

Komutların tamamı FinWallet.sln, compose.yml ve .env.example dosyalarının bulunduğu repository root dizininden çalıştırılmalıdır.

3. Local environment dosyasını oluştur

PowerShell:

```
Copy-Item .env.example .env
```

Bash:

```
cp .env.example .env
```

Ne yapar:

- Version control altında bulunan güvenli şablondan local .env üretir.
- .env .gitignore içindedir; commit edilmez.

.env içinde en az şu değerleri güçlü local değerlerle değiştir:

```
MSSQL_SA_PASSWORD
```

```
REDIS_PASSWORD
JWT_SIGNING_KEY
REGISTRATION_OTP_PEPPER
INTERNAL_SERVICE_KEY
DOWNSTREAM_SERVICE_KEY
```

Production secret'larını .env ile yönetme. Production'da orchestrator/secret-store injection kullan.

4. Compose dosyasını çalıştırmadan önce validate et

```
docker compose --env-file .env config --quiet
```

Ne yapar:

- Environment substitution yapar.
- YAML/Compose syntax'ını doğrular.
- Eksik required environment variable varsa burada fail eder.
- Container başlatmaz ve veri değiştirmez.

Resolved configuration görmek için:

```
docker compose --env-file .env config
```

Dikkat: resolved output secret değerleri gösterebilir. CI loguna veya paylaşılabılır dosyaya yönlendirme.

Servis listesini görmek için:

```
docker compose --env-file .env config --services
```

5. Image'ları build et

```
docker compose --env-file .env build --pull
```

Ne yapar:

- Gateway, FinWallet.Api ve fake API image'larını multi-stage Dockerfile ile build eder.
- --pull, base .NET image'larının mevcut tag için güncel halini kontrol eder.
- NuGet restore Docker BuildKit cache kullanır.
- Container başlatmaz.
- MSSQL ve Redis runtime image'ları up sırasında ayrıca pull edilebilir.

Cache kullanmadan tamamen temiz build gerekirse:

```
docker compose --env-file .env build --pull --no-cache
```

Bu daha yavaştır; normal geliştirmede sürekli kullanılmamalıdır.

6. Tüm sistemi ayağa kaldır

```
docker compose --env-file .env up -d --build
```

Ne yapar:

- 1 Gerekli image'ları build/pull eder.
- 2 finwallet-data ve finwallet-backend networklerini oluşturur.
- 3 MSSQL ve Redis named volume'larını oluşturur veya mevcut volume'ları tekrar bağlar.
- 4 MSSQL healthcheck başarılı olana kadar bekler.
- 5 mssql-init container'ı FinWallet DB'sini ve uygulanmamış schema versionlarını oluşturur.
- 6 Redis healthcheck başarılı olduktan sonra FinWallet.Api başlar.
- 7 Fake provider'lar ve Gateway başlar.
- 8 -d nedeniyle terminali bloklamadan detached modda çalışır.

İlk build/pull sonraki başlatmalardan daha uzun sürer.

7. Container durumlarını kontrol et

```
docker compose --env-file .env ps
```

Beklenen durum:

- mssql: Up / healthy
- redis: Up / healthy
- mssql-init: Exited (0)
- API ve Gateway servisleri: Up

mssql-init servisinin Exited (0) olması hata değildir. One-shot migration işi tamamlandığı için container kapanır.

Tüm container'ları Docker seviyesinde görmek için:

```
docker ps --filter name=finwallet
```

8. Gateway health endpoint'ini kontrol et

```
curl http://localhost:8080/health/live
```

Windows PowerShell:

```
Invoke-RestMethod http://localhost:8080/health/live
```

Gateway cevap veriyorsa host -> Gateway network path'i çalışıyor demektir.

Swagger Development/Docker environment'ta açıksa:

```
http://localhost:8080/swagger
```

Normal client API çağrıları Gateway <http://localhost:8080> adresine yapılmalıdır.

9. Logları izle

Tüm stack:

```
docker compose --env-file .env logs -f
```

Gateway:

```
docker compose --env-file .env logs -f gateway
```

FinWallet API:

```
docker compose --env-file .env logs -f finwallet-api
```

MSSQL:

```
docker compose --env-file .env logs -f mssql
```

Redis:

```
docker compose --env-file .env logs -f redis
```

Son 200 satır:

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api
```

-f yeni logları terminalde takip eder. Ctrl+C yalnız log takibini bitirir; detached container'ları durdurmaz.

Docker json-file log driver için rotation tanımlıdır. Uygulama loglarını container filesystem'inde kalıcı dosya olarak tutmak yerine stdout/stderr tercih edilmiştir.

10. MSSQL bağlantısını doğrula

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -d FinWallet -Q "SELECT DB_NAME() AS
DatabaseName; SELECT Version, AppliedAt FROM dbo.SchemaVersions ORDER BY Version;"
,
```

Ne yapar:

- SQL Server container'ının içinden DB'ye bağlanır.
- FinWallet DB'sinin açıldığını doğrular.
- 001, 002, 003 migration versionlarını gösterir.

Migration'ları tekrar kontrol etmek/eksik olanı uygulamak için:

```
docker compose --env-file .env run --rm mssql-init
```

SchemaVersions kaydı bulunan migration tekrar çalıştırılmaz.

11. Redis bağlantısını doğrula

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning ping'
```

Beklenen cevap:

PONG

Redis persistence bilgisi:

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning INFO persistence'
```

12. Debug portlarını aç

Normal compose security gereği backend servislerini host'a publish etmez. Local debugging gerektiğinde overlay kullan:

```
docker compose --env-file .env -f compose.yml -f compose.debug.yml up -d --build
```

Bu durumda localhost üzerinde:

Gateway	8080
FinWallet.Api	8081
FakeBank.Api	8082
FakeFraud.Api	8083
FakeCutoff.Api	8084
FakeCampaign.Api	8085
FakeCommunication.Api	8086
MSSQL	1433
Redis	6379

Portlar 127.0.0.1 bind edilir; LAN'a açılmaz.

Normal integration testinde yine Gateway kullan. Backend debug portları yalnız inspection/debugging içindir ve business endpointleri downstream service-key korumasını sürdürür.

13. Tek servisi rebuild/restart et

Örneğin FinWallet.Api kodu değişti:

```
docker compose --env-file .env build finwallet-api
docker compose --env-file .env up -d --no-deps finwallet-api
```

Ne yapar:

- Yalnız finwallet-api image'ını yeniden build eder.
- --no-deps MSSQL/Redis/Gateway gibi bağımlı servisleri yeniden yaratmaz.

Gateway için:


```
docker compose --env-file .env build gateway
docker compose --env-file .env up -d --no-deps gateway
```

Container'ı image rebuild etmeden restart etmek için:

```
docker compose --env-file .env restart gateway
```

14. Bir servisin shell'ine gir

```
docker compose --env-file .env exec finwallet-api /bin/sh
```

Runtime image non-root app user ile çalıştığı için root yetkisi bekleme.

Container environment'ını incelemek için secret'ları ekrana dökmemeye dikkat et.

15. Resource kullanımını izle

```
docker stats
```

Ne gösterir:

- CPU
- memory
- network I/O
- block I/O
- PID sayısı

Compose içinde application container'ları için CPU, memory ve PID limitleri tanımlıdır. Bu değerler local safety baseline'dır; production capacity planning ölçüm ile yapılmalıdır.

16. Volume'ları kontrol et

```
docker volume ls --filter name=finwallet
```

Detay:

```
docker volume inspect finwallet_mssql_data
docker volume inspect finwallet_mssql_backup
docker volume inspect finwallet_redis_data
```

Normal uygulama restart/recreate işlemlerinde volume'lar korunur.

17. SQL backup al

Önce backup oluştur:

```
docker compose --env-file .env exec mssql bash -lc '
SQLCMD=$(command -v /opt/mssql-tools18/bin/sqlcmd || command -v /opt/mssql-tools/bin/sqlcmd)
&&
$SQLCMD -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -Q "BACKUP DATABASE [FinWallet] TO
DISK = N''''/var/opt/mssql/backup/FinWallet.bak'''' WITH INIT, CHECKSUM"
'
```

Backup mssql_backup named volume içinde kalır.

Host'a kopyalamak istersen önce container ID al:

```
docker compose ps -q mssql
```

Bash örneği:

```
docker cp "$(docker compose ps -q mssql):/var/opt/mssql/backup/FinWallet.bak"
./FinWallet.bak
```

Backup dosyalarını Git'e commit etme.

18. Redis persistence snapshot iste

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
```

```
--no-auth-warning BGSAVE'
```

Bu komut Redis'e arka planda snapshot üretme talebi verir. Redis yine financial source of truth değildir; MSSQL backup'ın alternatifi değildir.

19. Servisleri geçici durdur ve tekrar başlat

Durdur:

```
docker compose --env-file .env stop
```

Başlat:

```
docker compose --env-file .env start
```

stop container'ları silmez. Volume ve container metadata korunur.

20. Stack'i kapat ama veriyi koru

```
docker compose --env-file .env down
```

Ne yapar:

- Compose container'larını siler.
- Compose networklerini siler.
- Named volume'ları silmez.
- Bir sonraki up MSSQL ve Redis verisini mevcut volume'lardan kullanır.

21. Tamamen sıfırla - DİKKAT veri siler

```
docker compose --env-file .env down -v --remove-orphans
```

Bu komut:

- container'ları siler;
- networkleri siler;
- mssql_data, mssql_backup, redis_data named volume'larını siler.

Sonuç: local DB, ledger, customer/auth data, Redis OTP state ve SQL backup volume'u kaybolur.

Yalnız gerçekten temiz local environment istediğinde kullan.

22. Yalnız Redis'i sıfırla

Önce stack'i durdur:

```
docker compose --env-file .env down
```

Sonra:

```
docker volume rm finwallet_redis_data
```

Tekrar:

```
docker compose --env-file .env up -d
```

MSSQL volume'u korunur.

23. Yalnız MSSQL'i sıfırla

```
docker compose --env-file .env down
docker volume rm finwallet_mssql_data finwallet_mssql_backup
docker compose --env-file .env up -d
```

Bu işlem financial data'yı tamamen siler. Local development dışında kullanılmamalıdır.

24. Kullanılmayan Docker disk alanını incele

```
docker system df
```

Build cache temizliği:

```
docker builder prune
```

Kullanılmayan image temizliği:

```
docker image prune
```

`docker system prune -a --volumes` çok daha agresiftir ve başka projelerin image/cache/volume'larını da etkileyebilir; rutin komut olarak kullanılmamalıdır.

25. Production-like overlay ile çalıştır

```
docker compose \
  --env-file .env \
  -f compose.yml \
  -f compose.production.yml \
  up -d --build
```

Bu overlay:

- ASPNETCORE_ENVIRONMENT=Production yapar;
- Swagger'ı kapatır;
- restart policy'yi sıkılaştırır;
- HSTS configuration'ını açar.

Bu dosya gerçek production platformunun yerine geçmez. Gerçek production için en az TLS ingress, external secret store, WAF/DDoS, image digest pinning, vulnerability scanning, backup/restore planı, monitoring/alerting ve network policy gerekir.

26. Docker Compose stack'i güncelle

Kod değişikliğinden sonra genel güncelleme:

```
git pull
docker compose --env-file .env build --pull
docker compose --env-file .env up -d
```

Schema scripti eklendiyse mevcut mssql-init scriptinin yeni migration'ı tanıdığı da güncellenmelidir. Migration'ı yalnız SQL dosyası ekleyip init listesine eklememek yeterli değildir.

27. Sık karşılaşılan problemler

Port 8080 kullanımda

```
docker compose --env-file .env ps
```

.env içinde:

```
GATEWAY_PORT=8090
```

yapıp yeniden `up -d` çalıştırabilirsin. Yeni Gateway adresi `http://localhost:8090` olur.

MSSQL unhealthy

```
docker compose --env-file .env logs --tail 300 mssql
```

Kontrol et:

- password SQL Server complexity koşullarını sağlıyor mu;
- yeterli Docker memory var mı;
- volume bozuk/uyumsuz mu;

- container sürekli restart ediyor mu.

mssql-init failed

```
docker compose --env-file .env logs mssql-init
```

Schema script error'ını düzelt. mssql-init one-shot olduğu için tekrar çalıştır:

```
docker compose --env-file .env run --rm mssql-init
```

Redis unhealthy

```
docker compose --env-file .env logs redis
```

Sonra:

```
docker compose --env-file .env exec redis sh -lc 'redis-cli -a "$REDIS_PASSWORD"
--no-auth-warning ping'
```

Gateway 502/503 dönüyor

```
docker compose --env-file .env logs --tail 200 gateway finwallet-api fake-bank fake-fraud
fake-cutoff fake-campaign fake-communication
```

Kontrol et:

- downstream servis Up mı;
- service DNS adı doğru mu;
- internal/downstream key eşleşiyor mu;
- Gateway health probe servisi unhealthy işaretlemiş mi.

Secret eksik

Compose şu formatta fail eder:

```
required variable ... is missing a value
```

.env dosyasının repository root'ta olduğundan ve required key'lerin boş olmadığından emin ol.

28. Minimum günlük geliştirme komut seti

İlk gün:

```
cp .env.example .env
docker compose --env-file .env config --quiet
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
docker compose --env-file .env logs -f gateway finwallet-api
```

Gün sonunda veriyi koruyarak kapat:

```
docker compose --env-file .env down
```

Ertesi gün:

```
docker compose --env-file .env up -d
docker compose --env-file .env ps
```

Tam local reset gerektiğinde:

```
docker compose --env-file .env down -v --remove-orphans
docker compose --env-file .env up -d --build
```

Son iki komutun local MSSQL/Redis datasını sildiğini unutma.